Research Notes

Prof. Floryan left me the following two papers (links). My goal is to learn about the model he is using for the fish to be able to implement it as code, then simplify this algorithm based on the Barnes-Hut algorithm.

1. Group cohesion and passive dynamics of a pair of inertial swimmers with three-dimensional hydrodynamic interactions.
2. Effects of symmetry and hydrodynamics on the cohesion of groups of swimmers.

Notes

1. Dr. Floryan's Automaton-Dipole Fish Model

1. The fish are essentially modelled as two point-masses connected by a rod of length ℓ.
2. There's a mathematical basis involving a "multipole expansion" (whatever that is) alongisde validations from other papers showing that using a source-sink pair separated by the length of the fish.
3. We orient them geometrically using symmetry to simplify them (essentially, we fix one of the fish and define the other fish relative to the first).
4. Their movement is (unsurprisingly) defined by a nonlinear first-order ODE.
5. The rest of the paper discusses the various resultant states that occur given different geometric configurations of the fish, as defined by three variables $\Delta\theta, \Delta x$, and Δy.
6. The fish move at a speed
$$U = \frac{\sigma}{4\pi\ell^2}$$
where $\sigma \geq 0$ is the volumetric flow rate of the source and sink.
7. Each 'swimmer' i has a center-of-mass position $\mathbf{x}_{c,i}$ and this has a separation between its source (head) at $\mathbf{x}_{f,i}$ and sink (tail) at $\mathbf{x}_{b,i}$ of ℓ along a direction denoted by the unit vector $\mathbf{n}_i$, related as:
$$\mathbf{x}_{f,i} = \mathbf{x}_{c,i} + \frac{1}{2}\ell\mathbf{n}_i$$
$$\mathbf{x}_{b,i} = \mathbf{x}_{c,i} - \frac{1}{2}\ell\mathbf{n}_i$$
this means that for a given time t, each swimmer's full orientation requires a minimum of five variables:
    1. The three-dimensional cartesian position of the fish, $\mathbf{x}_{c,i}$ (which is effectively three variables: $x_{c,i,x}, x_{c,i,y}$, and $x_{c,i,z}$).

2. The three-dimensional direction of the orientation unit vector $\mathbf{n}_i$, which can be reduced to two spherical dimensions ρ_i and θ_i knowing that the vector must always remain a unit vector.

8. The dynamics of each fish is defined based on the aforementioned 5 free variables alongside the self-propelled speed U to produce the self-propelled velocity of the source $\mathbf{v}_{f,i}$ as

$$\mathbf{v}_{f,i} = U\left(\mathbf{n}_i + \ell^2 \sum_{j\neq i}^{N} (\mathbf{r}_{f,i-f,j} - \mathbf{r}_{f,i-b,j})\right)$$

and the sink $\mathbf{v}_{b,i}$ as

$$\mathbf{v}_{b,i} = U\left(\mathbf{n}_i + \ell^2 \sum_{j\neq i} (\mathbf{r}_{b,i-f,j} - \mathbf{r}_{b,i-b,j})\right)$$

where

$$\mathbf{r}_{\alpha-\beta,i-j} = \frac{\mathbf{x}_{\alpha,i} - \mathbf{x}_{\beta,j}}{||\mathbf{x}_{\alpha,i} - \mathbf{x}_{\beta,j}||^3}$$

9. The translational ($\dot{\mathbf{x}}_{c,i}$) dynamics of the system is defined as

$$\dot{\mathbf{x}}_{c,i} = \frac{1}{2}(\mathbf{v}_{f,i} + \mathbf{v}_{b,i})$$

10. The rotational ($\dot{\mathbf{n}}_i$) dynamics of the system is defined as

$$\dot{\mathbf{n}}_i = \ell^{-1}(\mathbf{v}_{f,i} - \mathbf{v}_{b,i} + 2\lambda_i \mathbf{n}_i)$$

where

$$\lambda_i = -\frac{1}{2}[(\mathbf{v}_{f,i} - \mathbf{v}_{b,i}) \cdot \mathbf{n}_i]$$

is a Lagrange multiplier that ensures that ℓ is kept constant ($\dot{\ell} = 0$), and it originates

11. With all of this in mind, we can concatenate all of the position $\mathbf{x}_{c,i}$ and orientation vectors $\mathbf{n}_i$ of the system to produce the state vector for the whole system, $\mathbf{X}$, and this vector has dimension $6N$ (or $5N$ using spherical definition for $\mathbf{n}$).

12. The system is evolved forward in time using fourth-order Runge-Kutta (i.e., computing $\dot{\mathbf{X}}$ for each time t using a finite time-step δt based on computing $\dot{\mathbf{x}}_{c,i}$ and $\dot{\mathbf{n}}_i$).

2. The Barnes-Hut Algorithm

The fundamental problem of the project is that simulating large schools of fish is an n-body problem: the computational complexity (amount of time/compute required to calculate the motion) of the system scales on the order of n^2 ($\mathcal{O}(n^2)$), and systems of $n \geq 3$ are analytically unsolvable, requiring numerical simulation. This means that around $\log n \geq 2$ to 3 ($n \approx 100$ to 1000), these systems become too-complex to solve numerically as well (assuming we compute the differential evolution of each fish individually).

To solve this problem, we'll be employing the Barnes-Hut algorithm, which simplifies the system by clustering farther-out groups of fish within the school into a single conglomerate fish. This leads to increased (although marginal) error for a decrease in complexity to $\mathcal{O}(n \log n)$

Prof. Floryan left the following resources on the Barnes-Hut algorithm:

1. The Barnes-Hut Approximation: Efficient computation of N-body forces
2. Fast Hierarchical Methods for the N-body Problem, Part 1
3. The Barnes-Hut Algorithm
4. A hierarchical O(N log N) force-calculation algorithm

Given a spatial distribution of fish as described in the state matrix $\mathbf{X}$ defined as

$$\mathbf{X} = (\mathbf{X}_1, \mathbf{X}_2, \dots)$$

where

$$\mathbf{X}_i = [\mathbf{x}_{c,i}, \mathbf{n}_i]^T,$$

the three-dimensional Barnes-Hut algorithm applied to fish would work as follows:

1. We determine an iteration order for the fish (in this case, we'll use the natural listing order described in $\mathbf{X}$), and choose the first fish, $\mathbf{X}_1$, to be the root of the Barnes-Hut Octree, $\mathcal{O}_{\mathbf{X}}$.
2. We iterate over $\mathbf{X}$ and insert each fish $\mathbf{X}$ according to spatial orientation, meaning that we subdivide the three-dimensional space recursively until all of the fish in the tree are simplified.
3. We recursively calculate the center of mass for each cell (both leaves and greater nodes).
4. For each fish in the tree, traverse the tree to compute the force on the fish through deciding whether or not to use the center of mass or individual points based on how far the fish yielding a contribution to the force are.
5. Use these forces to calculate dynamic change and evolve the state numerically, then repeat from step 1 for the next time-step.

There is also this paper for generating Octrees:

Cornerstone: Octree Construction Algorithms for Scalable Particle Simulations

June 2, 2026

Yesterday, I began by trying to write out codes for performing the simulation. I spent about 6 hours working, and I was unable to complete the code. It was also object-oriented, and this is very slow and memory-consuming. What I'll be moving on to doing is switching to the `numba` library and building the base system today using a more mathematical array-based method.

I'd also like to rework the mathematical notation for the far-field fish model proposed by Dr. Floryan to simplify the Python code. As before, the fish are defined with length ℓ, a center-of-mass position $\mathbf{x}_{c,i}$, and orientation vector $\mathbf{n}_i$ in a body of water with volumetric flow rate σ. Together, the entire school of fish for any instant in time can be called the "system state" and represented with the vector $\mathbf{X}_t$. The fish move at a self-propelled speed

$$U = \frac{\sigma}{4\pi\ell^2}$$

and the front/head (source) $\mathbf{x}_{f,i}$ and back/tail (sink) $\mathbf{b}, \mathbf{i}$ of each fish are located at

$$\mathbf{x}_{f,i} = \mathbf{x}_{c,i} + \delta_i$$

and

$$\mathbf{x}_{b,i} = \mathbf{x}_{c,i} - \delta_i$$

respectively, where

$$\delta_i = \frac{1}{2}\ell\mathbf{n}_i.$$

Then, the head and the tail move at velocities $\mathbf{v}_{f,i}$ and $\mathbf{v}_{b,i}$ respectively, and each is defined as

$$\mathbf{v}_{\alpha,i} = U\mathbf{n}_i + \frac{\sigma}{4\pi}\sum_{j\neq i}^{N}\mathbf{h}_{\alpha,i,j}$$

where $\mathbf{h}_{\alpha,i,j}$ represents the *interaction vector* from fish j to fish i at α (the front or the back of the fish). The first term, $U\mathbf{n}_i$, represents the effect of the front and back (source and sink) of fish i on each other, and the second term represents the pairwise interactions between fish i and all other fish j, to both their sources and sinks.

An interaction vector is defined as the difference between two "Coulombic Force-Distance Vectors" (a general vector for the distance-contribution of the Coulomb-like hydrodynamic force), and the Coulombic force vector represents the mathematical drop-off of force strength by distance, with units of distance^{-2}. Each force-distance (FD) vector $\mathbf{c}_{\alpha\beta}$ is defined as

$$\mathbf{c}_{\alpha\beta} = \frac{\mathbf{d}_{\alpha\beta}}{||\mathbf{d}_{\alpha\beta}||^3}$$

where $\mathbf{d}_{\alpha\beta}$ is the displacement vector between feature α (front/back) of fish i and feature β of fish j, defined as

$$\mathbf{d}_{\alpha\beta} = \mathbf{x}_{\alpha,i} - \mathbf{x}_{\beta,j}$$

There are two types of interaction vectors: the front (or source) interaction vector $\mathbf{h}_{f,i,j}$ defined as

$$\mathbf{h}_{f,i,j} = \mathbf{c}_{ff} - \mathbf{c}_{fb}$$

and the back (or sink) interaction vector $\mathbf{h}_{b,i,j}$ defined as

$$\mathbf{h}_{b,i,j} = \mathbf{c}_{bf} - \mathbf{c}_{bb}.$$

We can prove that this formulation makes sense more rigorously, but it is easiest to understand by thinking of the source/sink interactions more generally and how they contribute to the final hydrodynamic force.

Then, we can compute the translational $\dot{\mathbf{x}}_{c,i}$ and rotational $\dot{\mathbf{n}}_i$ derivatives for each fish, the full set of which forms the full state derivative for our school, $\dot{\mathbf{X}}$, and this is

$$\dot{\mathbf{x}}_{c,i} = \frac{\mathbf{v}_{f,i} + \mathbf{v}_{b,i}}{2}$$

and

$$\dot{\mathbf{n}}_i = \frac{\Delta\mathbf{v}_{f-b,i} + 2\lambda_i \mathbf{n}_i}{\ell}$$

where λ_i is a lagrange multiplier that ensures that ℓ is kept constant

$$\lambda_i = \frac{-\Delta\mathbf{v}_{f-b,i} \cdot \mathbf{n}_i}{2}$$

and $\Delta\mathbf{v}_{f-b,i}$ is the difference in velocity between the head and tail

$$\Delta\mathbf{v}_{f-b,i} = \mathbf{v}_{f,i} - \mathbf{v}_{b,i}.$$

Runge-Kutta Fourth Order

Fourth-order Runge Kutta (RK4) is defined for ordinary differential equations of the form

$$y'(t) = f(y, t)$$

given a time-step δt and subsequent half-step $\delta t_{1/2} = \delta t/2$, and is calculated by computing the following values, starting from $y(t_0) = y_0$:

$$\begin{aligned}
y_1' &= f(y_0, t_0) & y_1 &= y_0 + y_1' \delta t_{1/2} \\
y_2' &= f(y_1, t_0 + \delta t_{1/2}) & y_2 &= y_0 + y_2' \delta t_{1/2} \\
y_3' &= f(y_2, t_0 + \delta t_{1/2}) & y_3 &= y_0 + y_3' \delta t \\
y_4' &= f(y_3, t_0 + \delta t)
\end{aligned}$$

Then, the final derivative for $t = t_0$ is approximated using the weighted sum of each of these points

$$y_{t_0}' = \frac{y_1' + 2y_2' + 2y_3' + y_4'}{6}.$$

The final value can then be approximated best as

$$y(t_0 + \delta t) \approx y_0 + y_{t_0}' \delta t.$$

RK4, Applied to the Project

RK4 is best applied for this project by treating $\mathbf{X}$ as y and the process described above as $f(\mathbf{X}, t)$.

June 3

Today, I started out with Mohamed telling me about why my previous calculation,

$$\mathbf{v}_{\alpha,i} = U \left(\hat{n}_i + \ell^2 \sum_{j \neq i}^{N} \mathbf{h}_{\alpha,ij} \right)$$

was incorrect. As it turns out, σ is normally a property associated with each fish, but we assumed that ℓ and σ are the same for each fish. In a more complex/realistic model, each fish $\mathbf{s}_i$ would be described by the parameters

$$\mathbf{s}_i := (\sigma_i, \ell_i, \mathbf{x}_{c,i}(t), \hat{n}_i(t)) \tag{1}$$

where $\mathbf{x}_c$ and $\hat{n}$ are both time-dependent, and the full state is defined as

$$\mathbf{X} := [\mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_N]. \tag{2}$$

Then, the self-propelled speed U_i is a property of each fish, calculated with

$$U_i = \frac{\sigma_i}{4\pi\ell^2} \tag{3}$$

and the velocity of a feature α is computed as

$$\mathbf{v}_{\alpha,i} = U_i \hat{n}_i + \sum_{j \neq i}^{N} \frac{\sigma_j}{4\pi} \mathbf{h}_{\alpha,ij} \tag{4}$$

where $\mathbf{h}_{\alpha,ij}$ is the interaction vector for feature α between fish i and j. Interaction vectors have units of $1/\text{distance}^2$ and are different for the sources and sinks.